



Chap05

함수와 참조, 복사 생성자

함수의 인자 전달 방식 리뷰

□ 인자 전달 방식

▣ 값에 의한 호출, call by value

- 함수가 호출되면 매개 변수가 스택에 생성됨
- 호출하는 코드에서 값을 넘겨줌
- 호출하는 코드에서 넘어온 값이 매개 변수에 복사됨

▣ 주소에 의한 호출, call by address

▣ 함수의 매개 변수는 포인터 타입

- 함수가 호출되면 포인터 타입의 매개 변수가 스택에 생성됨
- 호출하는 코드에서는 명시적으로 주소를 넘겨줌
 - 기본 타입 변수나 객체의 경우, 주소 전달
 - 배열의 경우, 배열의 이름
- 호출하는 코드에서 넘어온 주소 값이 매개 변수에 저장됨

값에 의한 호출과 주소에 의한 호출



(a) 값에 의한 호출



(b) 주소에 의한 호출

그림5-2(a)

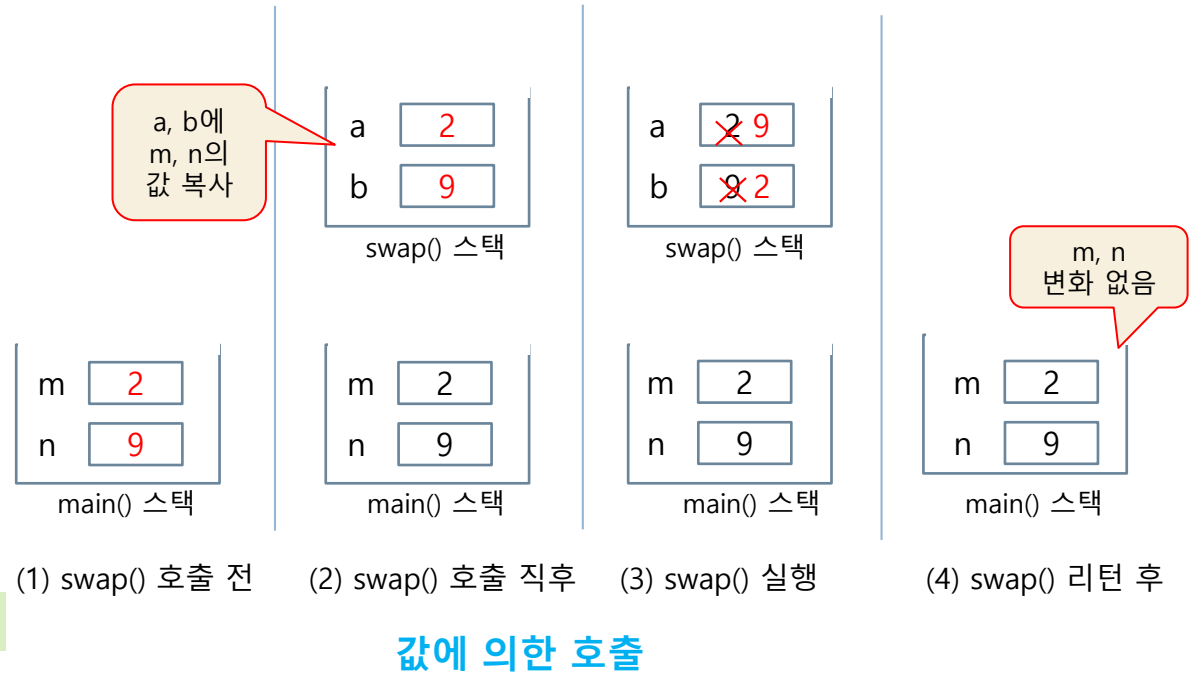
```
#include <iostream>
using namespace std;

void swap(int a, int b) {
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int m=2, n=9;
    swap(m, n);
    cout << m << " " << n;
}
```

2 9



* 값에 의한 호출은 실인자의 값을 복사하여 전달하므로 함수내에서 실인자를 손상시킬수 없는 장점이 있다.
즉 함수 호출에 따른 부작용은 없다.

* 값에 의한 호출은 실인자 객체의 크기가 크면 객체를 복사하는 시간이 커지는 단점이 있다.

그림5-2(b)

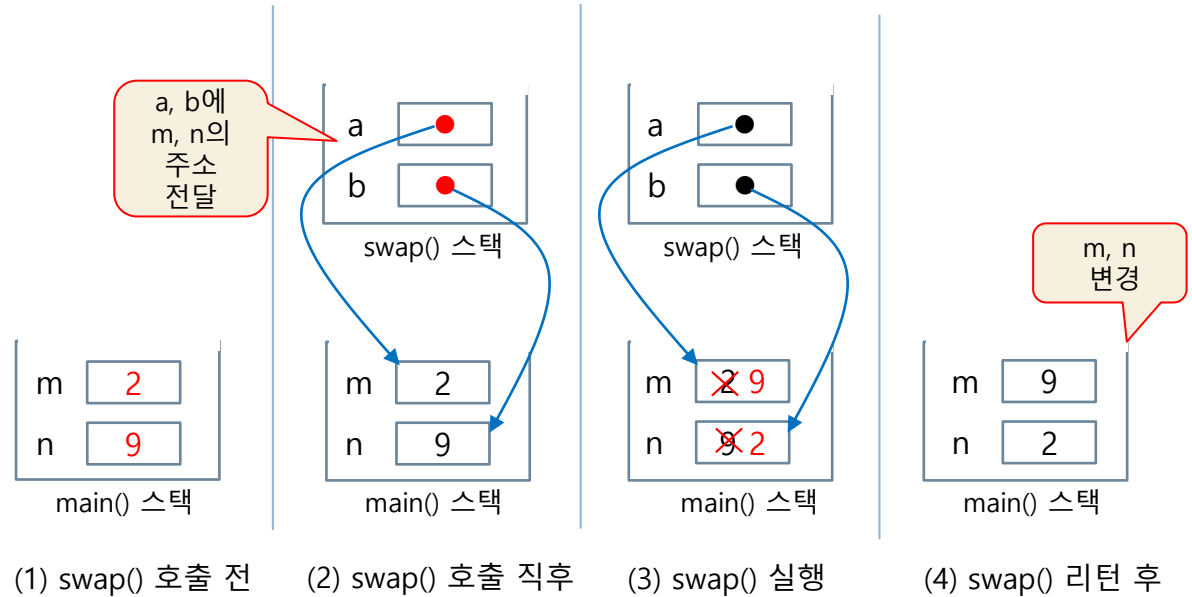
```
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main() {
    int m=2, n=9;
    swap(&m, &n);
    cout << m << " " << n;
}
```

9 2



주소에 의한 호출

* 주소에 의한 호출은 실인자의 **주소**를 넘겨주어 의도적으로 함수내에서 실인자의 값을 변경하고자 할때 사용한다.

```

#include <iostream>
using namespace std;

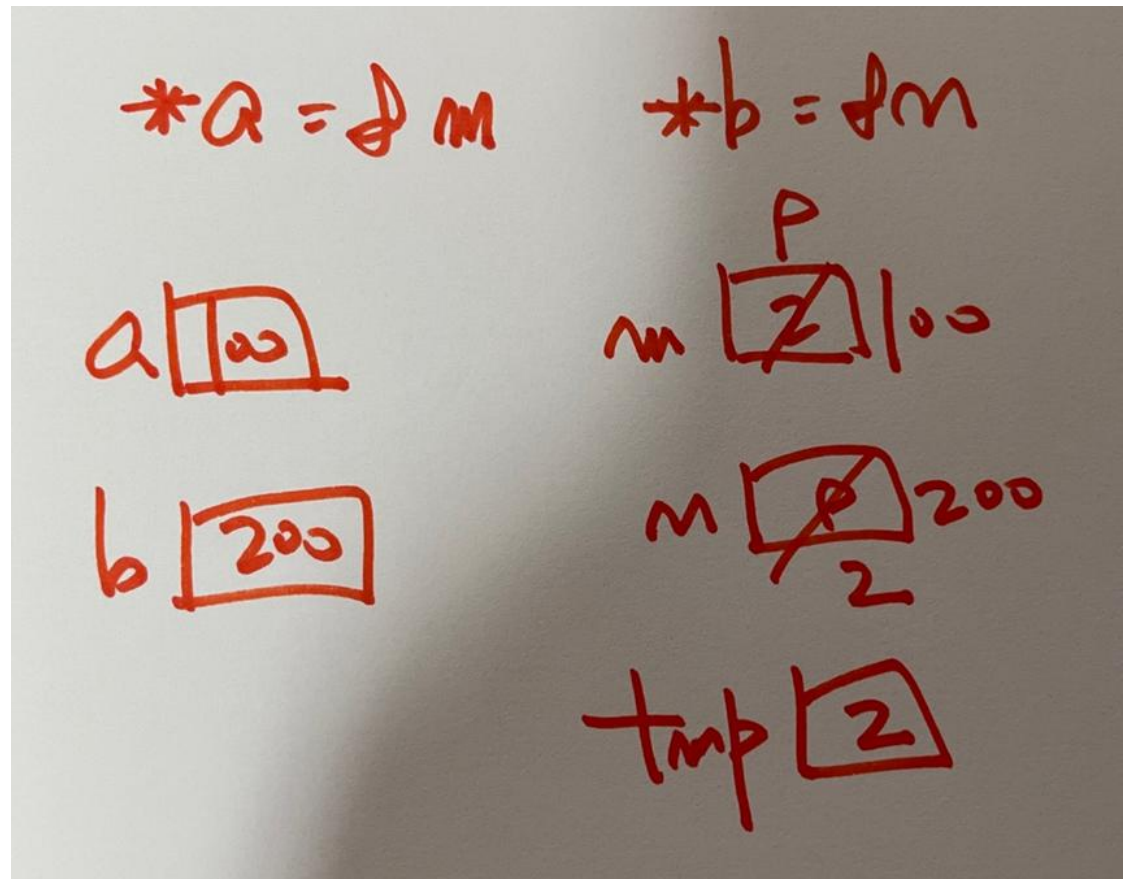
void swap(int *a, int *b) {
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main() {
    int m=2, n=9;
    swap(&m, &n);
    cout << m << ' ' << n;
}

```

9 2



'값에 의한 호출'로 객체 전달 call by value

7

- 함수를 호출하는 쪽에서 객체 전달
 - 객체 이름만 사용
 - 함수의 매개 변수 객체 생성
 - 매개 변수 객체의 공간이 스택에 할당
 - 호출하는 쪽의 객체가 매개 변수 객체에 그대로 복사됨
 - 매개 변수 객체의 생성자는 호출되지 않음
 - 함수 종료
 - 매개 변수 객체의 소멸자 호출
- 매개 변수 객체의 생성자 소멸자의 비대칭 실행 구조
- 값에 의한 호출 시 매개 변수 객체의 생성자가 실행되는것이 아니고 복사 생성자가 실행됨
 - 호출되는 순간의 실인자 객체 상태를 매개 변수 객체에 그대로 전달하기 위함 // 복사생성자

'값에 의한 호출' 방식으로 increase(Circle c) 함수가 호출되는 과정

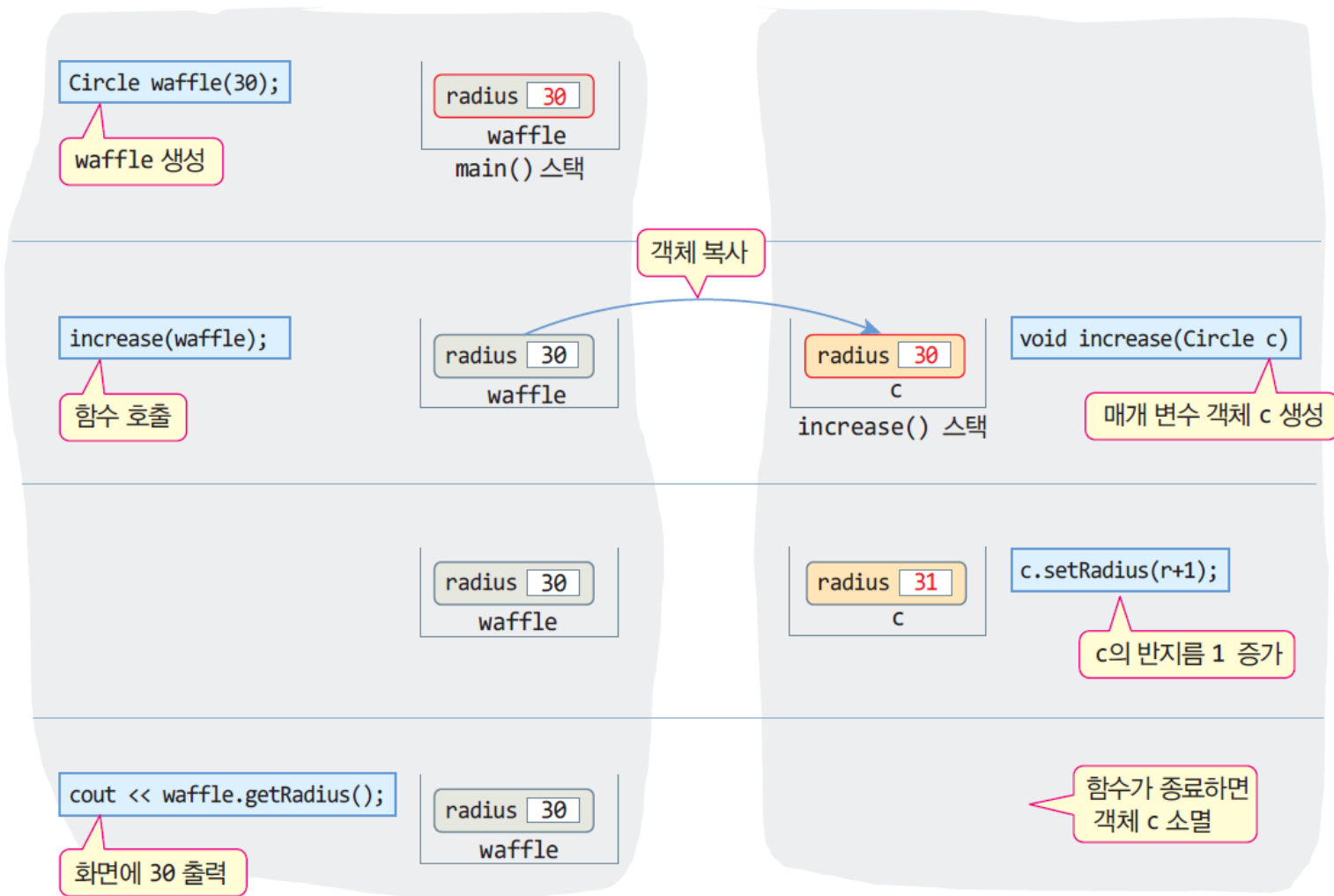
→ 실행 결과

30

```
int main() {  
    Circle waffle(30);  
    increase(waffle);  
    cout << waffle.getRadius() << endl;  
}
```

call by value

```
void increase(Circle c) {  
    int r = c.getRadius();  
    c.setRadius(r+1);  
}
```



예제 5-1 '값에 의한 호출'시 객체를 매개 변수로 가지는 생성자는 실행되지 않음

```
#include <iostream>
using namespace std;

class Circle {
private:
    int radius;
public:
    Circle();
    Circle(int r);
    ~Circle();
    double getArea() { return 3.14*radius*radius; }
    int getRadius() { return radius; }
    void setRadius(int radius) { this->radius = radius;};

    Circle::Circle() {
        radius = 1;
        cout << "생성자 실행 radius = " << radius << endl;
    }

    Circle::Circle(int radius) {
        this->radius = radius;
        cout << "생성자 실행 radius = " << radius << endl;
    }

    Circle::~Circle() {
        cout << "소멸자 실행 radius = " << radius << endl;
    }
}
```

waffle을 복사해
새 객체 c 생성

```
void increase(Circle c) {
    int r = c.getRadius();
    c.setRadius(r+1);
}

int main() {
    Circle waffle(30);
    increase(waffle);
    cout << waffle.getRadius() << endl;
}
```

waffle의 내용이
그대로 c에 복사

waffle 생성

생성자 실행 radius = 30

소멸자 실행 radius = 31

30

소멸자 실행 radius = 30

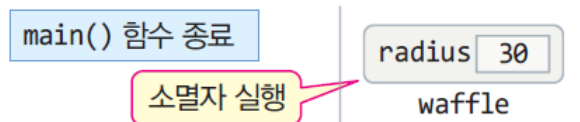
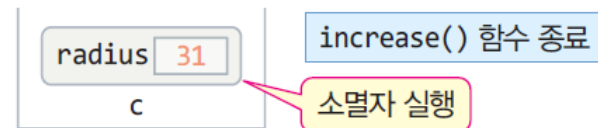
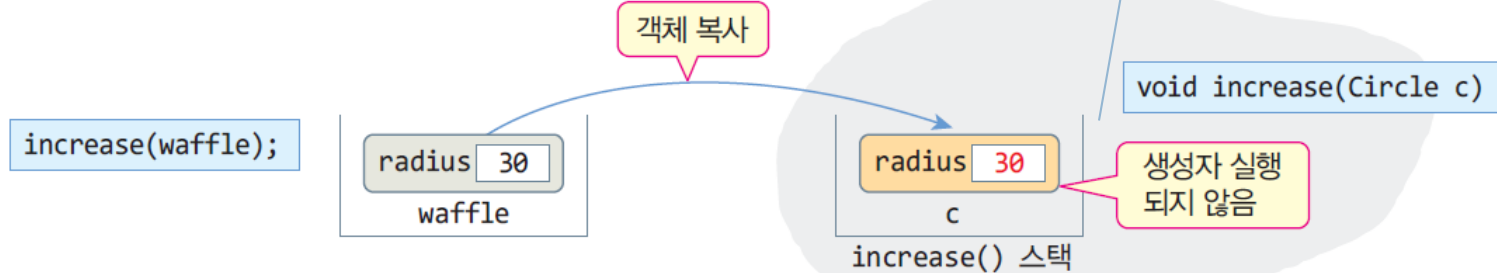
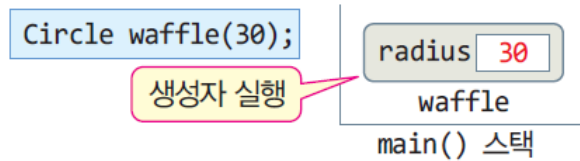
waffle 소멸

c의 생성자
실행되지 않았음

c 소멸

'값에 의한 호출'시에 생성자와 소멸자의 비대칭 실행

정확하게는 복사생성자가 실행하는데, 복사생성자 출력코드가 없어 생성자가 실행하지 않는것 처럼 보임



```

#include <iostream>
using namespace std;

class Circle {
private:
    int radius;
public:
    Circle();
    Circle(int r);
    Circle(const Circle& c);
    ~Circle();
    double getArea() { return 3.14*radius*radius; }
    int getRadius() { return radius; }
    void setRadius(int radius) { this->radius = radius; }
};

Circle::Circle() {
    radius = 1;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::Circle(int radius) {
    this->radius = radius;
    cout << "생성자 실행 radius = " << radius << endl;
}

```

```

Circle::Circle(const Circle& c) {
    radius = c.radius;
    cout << "복사 생성자 실행 radius = " << radius << endl;
}

```

```

Circle::~Circle() {
    cout << "소멸자 실행 radius = " << radius << endl;
}

```

```

void increase(Circle c) {
    int r = c.getRadius();
    c.setRadius(r+1);
}

```

```

int main() {
    Circle waffle(30);
    increase(waffle);
    cout << waffle.getRadius() << endl;
}

```

Microsoft Visual Studio 디버그

```

생성자 실행 radius = 30
복사 생성자 실행 radius = 30
소멸자 실행 radius = 31
30
소멸자 실행 radius = 30

```

```
Microsoft Visual Studio 디버그 × +
생성자 실행 radius = 30
복사 생성자 실행 radius = 30
소멸자 실행 radius = 31
30
소멸자 실행 radius = 30
```

복사 생성자는 "복사되는 순간의 값"을 출력 => 30
소멸자는 "죽기 직전의 값"을 출력 => 31

[복사 순간]

c.radius = 30 → 복사 생성자 출력 (30)

[값 변경]

c.radius = 31

[소멸 순간]

c.radius = 31 → 소멸자 출력 (31)

복사 생성자는 객체 생성 시점의 상태를 출력하고,
소멸자는 객체 소멸 직전의 상태를 출력하므로 값이 달라질 수 있다

```

Circle::Circle(const Circle& c) {
    radius = c.radius;
    cout << "복사 생성자 실행 radius = " << radius << endl;
}

```

구분	일반 생성자	복사 생성자
형태	Circle(int r)	Circle(const Circle& c)
역할	새 객체 생성	기존 객체 복사
매개변수	값	객체(참조)
& 사용	×	○

함수에 객체 전달 - '주소에 의한 호출'로

call by address

- 함수 호출시 객체의 **주소** 만 전달
 - ▣ 함수의 매개 변수는 객체에 대한 포인터 변수로 선언
 - ▣ 함수 호출 시 생성자 소멸자가 실행되지 않는 구조

주소에 의한 호출로 객체 전달(그림 5-5)

```
1  #include <iostream>
2  using namespace std;
3
4  class Circle {
5  private:
6      int radius;
7  public:
8      Circle();
9      Circle(int r);
10     ~Circle();
11     double getArea() { return 3.14*radius*radius; }
12     int getRadius() { return radius; }
13     void setRadius(int radius) { this->radius = radius; }
14 };
15
16 Circle::Circle() {
17     radius = 1;
18     cout << "생성자 실행 radius = " << radius << endl;
19 }
```

주소에 의한 호출로 객체 전달(그림 5-5)

```
21  ◀ Circle::Circle(int radius) {  
22      |      this->radius = radius;  
23      |      cout << "생성자 실행 radius = " << radius << endl;  
24      |  }  
25  
26  ◀ Circle::~~Circle() {  
27      |      cout << "소멸자 실행 radius = " << radius << endl;  
28      |  }  
29  
30  ◀ void increase(Circle *p) {  
31      |      int r = p->getRadius();  
32      |      p->setRadius(r+1);  
33      |  }  
34  
35  ◀ int main() {  
36      |      Circle waffle(30);  
37      |      increase(&waffle);  
38      |      cout << waffle.getRadius() << endl;  
39      |  }
```

포인터p는 객체가 아
니므로 생성자나 소멸
자와 상관이 없다

```
생성자 실행 radius = 30  
31  
소멸자 실행 radius = 31
```


'주소에 의한 호출'로 increase(Circle *p) 함수가 호출되는 과정

31

```
int main() {  
    Circle waffle(30);  
    increase(&waffle);  
    cout << waffle.getRadius() ;  
}
```

call by address

```
void increase(Circle *p) {  
    int r = p->getRadius();  
    p->setRadius(r+1);  
}
```

Circle waffle(30);

waffle 생성

radius 30

waffle
main() 스택

waffle의 주소가
p에 전달

increase(&waffle);

함수호출

radius 30

waffle

p

increase() 스택

void increase(Circle *p)

매개 변수 포인터 p 생성

radius 31

waffle

p

p->setRadius(r+1);

waffle의 반지름 1 증가

cout << waffle.getRadius();

31이 화면에 출력됨

radius 31

waffle

함수가 종료하면
포인터 p 소멸

객체 치환 및 객체 리턴

□ 객체 치환(기존 값을 다른 값으로 바꾸는 것)

- 동일한 클래스 타입의 객체끼리 치환 가능
- 객체의 모든 데이터가 **비트 단위** 로 복사

```
Circle c1(5);  
Circle c2(30);  
c1 = c2; // c2 객체를 c1 객체에 비트 단위 복사. c1의 반지름 30됨
```

- 치환된 두 객체는 현재 내용물만 같을 뿐 독립적인 공간 유지

객체 치환 및 객체 리턴

□ 객체 리턴

▣ 객체의 복사본 리턴

```
Circle getCircle() {  
    Circle tmp(30);  
    return tmp; // 객체 tmp 리턴  
}
```

```
Circle c; // c의 반지름 1  
c = getCircle(); // tmp 객체의 복사본이 c에 치환. c의 반지름은 30이 됨
```

예제 5-2 객체 리턴

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle() { radius = 1; }
    Circle(int radius) { this->radius = radius; }
    void setRadius(int radius) { this->radius = radius; }
    double getArea() { return 3.14*radius*radius; }
};

Circle getCircle() {
    Circle tmp(30);
    return tmp; // 객체 tmp를 리턴한다.
}

int main() {
    Circle c; // 객체가 생성된다. radius=1로 초기화된다.
    cout << c.getArea() << endl;

    c = getCircle();
    cout << c.getArea() << endl;
}
```

tmp 객체의 복사본이
리턴된다.

tmp 객체가 c에 복사된다.
c의 radius는 30이 된다.

참조란?



메뚜기는 유재석의 별명



참조(reference)란 가리킨다는 뜻으로,
이미 존재하는 객체나 변수에 대한 별명

참조 활용

- 참조 변수
- 참조에 의한 호출
- 참조 리턴

참조란?



참조 변수

- 참조 변수 선언
 - ▣ 참조자 &의 도입
 - ▣ 이미 존재하는 변수에 대한 다른 이름(별명)을 선언
 - 참조 변수는 이름만 생기며
 - 참조 변수에 새로운 공간을 할당하지 않는다.
 - 초기화로 지정된 기존 변수를 공유한다.

```
int n=2;  
int &refn = n; // 참조 변수 refn 선언. refn은 n에 대한 별명
```

```
Circle circle;  
Circle &refc = circle; // 참조 변수 refc 선언. refc는 circle에 대한 별명
```



참조와 포인터

-포인터

포인터는 & 연산자를 사용해 변수의 주소를 저장하는 변수

예: `int *p = &a;`

포인터는 주소에 직접 접근해 값을 읽거나 수정함

`*p = 10;`처럼 포인터 연산자(*)를 사용해 주소에 저장된 값을 변경

-참조

참조는 & 연산자 없이 변수명만으로 주소를 저장하는 변수

예: `int &r = a;`

참조는 변수 자체와 동일한 메모리 위치를 공유하며,

참조 변수를 통해 값을 직접 변경함

단, 참조는 초기화 시에만 주소가 할당되며, 이후에는 주소를 변경할 수 없음

-차이점

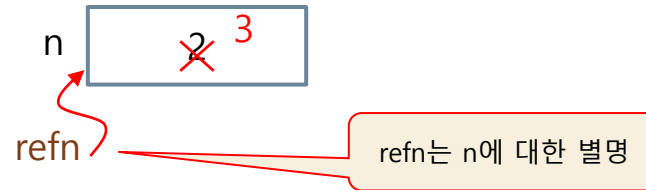
포인터: 주소를 저장·변경 가능, & 연산자 사용, NULL 값 가능, 중첩 포인터 가능

참조: 변수명을 통한 간접 접근, 초기화만 가능, 주소 변경 불가, NULL 불가

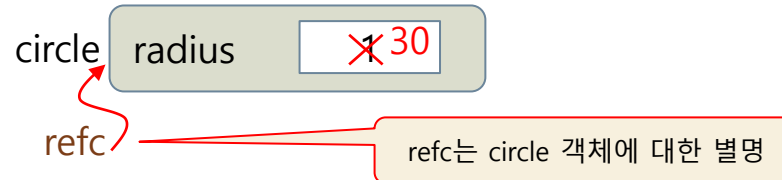
이처럼 포인터는 주소와 값 모두를 다루고, 참조는 값만을 다루며 선언 방식이 다름

참조 변수 선언 및 사용 사례

```
int n = 2;  
int &refn = n;  
  
refn = 3;
```



```
Circle circle;  
Circle &refc = circle;  
  
refc.setRadius(30);
```



refc->setRadius(30);으로 하면 안 됨

예제 5-3 기본 타입 변수에 대한 참조

```
#include <iostream>
using namespace std;

int main() {
    cout << "i" << '\t' << "n" << '\t' << "refn" << endl;
    int i = 1;
    int n = 2;
    int &refn = n; // 참조 변수 refn 선언. refn은 n에 대한 별명
    n = 4;
    refn++; // refn=5, n=5
    cout << i << '\t' << n << '\t' << refn << endl;

    refn = i; // refn=1, n=2
    refn++; // refn=2, n=2
    cout << i << '\t' << n << '\t' << refn << endl;

    int *p = &refn; // p는 n의 주소를 가짐
    *p = 20; // refn=20, n=20
    cout << i << '\t' << n << '\t' << refn << endl;
}
```

참조 변수 refn 선언

참조에 대한
포인터 변수 선언

i	n	refn
1	5	5
1	2	2
1	20	20

예제 5-4 객체에 대한 참조

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle() { radius = 1; }
    Circle(int radius) { this->radius = radius; }
    void setRadius(int radius) { this->radius = radius; }
    double getArea() { return 3.14*radius*radius; }
};

int main() {
    Circle circle;
    Circle &refc = circle;
    refc.setRadius(10);
    cout << refc.getArea() << " " << circle.getArea();
}
```

circle 객체에 대한
참조 변수 refc 선언

참조에 의한 호출

- 참조를 가장 많이 활용하는 사례
- **call by reference** 라고 부름
- 함수 형식
 - ▣ 함수의 매개 변수를 참조 타입으로 선언
 - 참조 매개 변수(reference parameter)라고 부름
 - 참조 매개 변수는 실인자 변수를 참조함
 - 참조매개 변수의 이름만 생기고 공간이 생기지 않음
 - 참조 매개 변수는 실인자 변수 공간 공유
 - 참조 매개 변수에 대한 조작은 실인자 변수 조작 효과

call by value, reference, address

Call by value (값에 의한 호출)

함수에 전달된 변수의 복사본이 생성되어, 함수 내부에서 값을 바꿔도 원래 변수에는 영향이 없음
주로 기본형 변수에 사용되며, 값만 전달됩니다.

Call by reference (참조에 의한 호출)

함수에 변수의 참조(주소)를 전달해, 함수 내에서 원본 변수를 직접 수정할 수 있음
주로 포인터나 참조형 변수에 사용되며, 변경이 호출자에게 반영됨.

Call by address (주소 전달)

함수에 변수의 주소를 전달하는 방식으로, Call by reference와 유사하게 동작함
함수 내에서 원본 변수를 직접 수정할 수 있으며, 주로 포인터 인자 전달 시 사용됨

결론

Call by value는 값 복사, Call by reference/Call by address는 참조(주소) 전달로, 함수 내에서 변수 수정 시 어느 방식이냐에 따라 결과가 달라짐.

참조에 의한 호출 사례

```
#include <iostream>
using namespace std;
```

```
void swap(int &a, int &b) {
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}
```

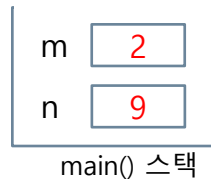
```
int main() {
    int m=2, n=9;
    swap(m, n);
    cout << m << ' ' << n;
}
```

참조 매개 변수 a, b

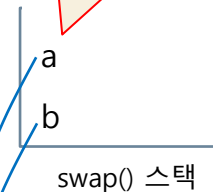
참조 매개 변수를
보통 변수처럼 사용

함수가 호출되면
m, n에 대한 참
조 변수 a, b가
생긴다.

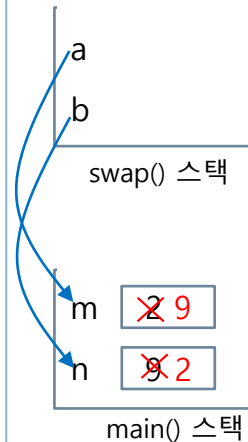
a, b는 m, n의 별명.
a, b 이름만 생성.
변수 공간 생기지
않음



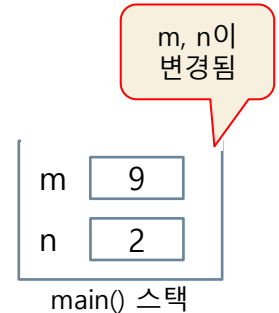
(1) swap() 호출 전



(2) swap() 호출 직후



(3) swap() 실행



(4) swap() 리턴 후

m, n이
변경됨

9 2

참조 매개변수가 필요한 사례

- 다음 코드에 어떤 문제가 있을까?
 - ▣ average() 함수의 작동
 - 계산에 오류가 있으면 0 리턴, 아니면 평균 리턴
 - ▣ 만일 average()가 리턴한 값이 0이라면?
 - 평균이 0인 거야? 아니면 오류가 발생한 거야?

```
int average(int a[], int size) {  
    if(size <= 0) return 0;  
    int sum = 0;  
    for(int i=0; i<size; i++) sum += a[i];  
    return sum/size;  
}
```

```
int x[ ]={1,2,3,4};  
int avg = average(x, 4);  
  
// avg는 2
```

흠, 평균이 2군.
알았어!

```
int x[ ]={1,2,3,4};  
int avg = average(x, -1);  
  
// avg는 0
```

평균이 0인 거야,
아니면 오류가 난 거야?

예제 5-5에서 해결

예제 5-5 참조 매개 변수로 평균 리턴하기

참조 매개 변수를 통해 평균을 리턴하고 리턴문을 통해서 함수의 성공 여부를 리턴하도록 average() 함수를 작성하라

```
#include <iostream>
using namespace std;
```

```
bool average(int a[], int size, int& avg) {
```

```
    if(size <= 0)
```

```
        return false;
```

```
    int sum = 0;
```

```
    for(int i=0; i<size; i++)
```

```
        sum += a[i];
```

```
    avg = sum/size;
```

```
    return true;
```

```
}
```

```
int main() {
```

```
    int x[] = {0,1,2,3,4,5};
```

```
    int avg;
```

```
    if(average(x, 6, avg)) cout << "평균은 " << avg << endl;
```

```
    else cout << "매개 변수 오류" << endl;
```

```
    if(average(x, -2, avg)) cout << "평균은 " << avg << endl;
```

```
    else cout << "매개 변수 오류 " << endl;
```

```
}
```

참조 매개 변수 avg에
평균 값 전달

avg에 평균이 넘어오고,
average()는 true 리턴

avg의 값은 의미없고,
average()는 false 리턴

평균은 2
매개 변수 오류

예제 5-6 참조에 의한 호출로 Circle 객체에 참조 전달

```
#include <iostream>
using namespace std;

class Circle {
private:
    int radius;
public:
    Circle();
    Circle(int r);
    ~Circle();
    double getArea() { return 3.14*radius*radius; }
    int getRadius() { return radius; }
    void setRadius(int radius) { this->radius = radius; }
};

Circle::Circle() {
    radius = 1;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::Circle(int radius) {
    this->radius = radius;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::~~Circle() {
    cout << "소멸자 실행 radius = " << radius << endl;
}
```

참조 매개 변수 c

```
void increaseCircle(Circle &c) {
    int r = c.getRadius();
    c.setRadius(r+1);
}

int main() {
    Circle waffle(30);
    increaseCircle(waffle);
    cout << waffle.getRadius() << endl;
}
```

참조에 의한 호출

waffle 객체 생성

생성자 실행 radius = 30
31
소멸자 실행 radius = 31

waffle 객체 소멸

*참조 매개 변수로 이루어진 모든 연산은 원본 객체에 대한 연산이 된다
*참조 매개 변수는 이름만 생성되므로 생성자와 소멸자는 생성되지 않는다.

참조 리턴

□ C 언어의 함수 리턴

▣ 함수는 반드시 값만 리턴

- 기본 타입 값 : int, char, double 등
- 포인터 값

□ C++의 함수 리턴

▣ 함수는 값 외에 참조 리턴 가능

▣ 참조 리턴

- 변수 등과 같이 현존하는 공간에 대한 참조 리턴
 - 변수의 값을 리턴하는 것이 아님

값을 리턴하는 함수 vs. 참조를 리턴하는 함수

문자
리턴

```
char c = 'a';

char get() { // char 리턴
    return c; // 변수 c의 문자('a') 리턴
}

char a = get(); // a = 'a'가 됨

get() = 'b'; // 컴파일 오류
```

char 타입
의 공간에
대한 참조
리턴

```
char c = 'a';

char& find() { // char 타입의 참조 리턴
    return c; // 변수 c에 대한 참조 리턴
}

char a = find(); // a = 'a'가 됨

char &ref = find(); // ref는 c에 대한 참조
ref = 'M'; // c = 'M'

find() = 'b'; // c = 'b'가 됨
```

find()가 리턴한 공
간에 'b' 문자 저장

(a) 문자 값을 리턴하는 get()

(b) char 타입의 참조(공간)을 리턴하는 find()

get()는 이름도 없고 저장된 변수가
아닌 임시 복사본으로 값대입 불가
능

find()는 c의 참조이므로
함수호출 결과가 변수처럼 사용됨

예제 5-8 간단한 참조 리턴 사례

```
#include <iostream>
using namespace std;
```

```
char& find(char s[], int index) {
    return s[index]; // 참조 리턴
}
```

```
int main() {
    char name[] = "Mike";
    cout << name << endl;
```

```
    find(name, 0) = 'S'; // name[0]='S'로 변경
    cout << name << endl;
```

```
    char& ref = find(name, 2);
    ref = 't'; // name = "Site"
    cout << name << endl;
```

```
}
```

s[index] 공간의 참조 리턴

find()가 리턴한 위치에 문자 'm' 저장

ref는 name[2] 참조

(1) `char name[] = "Mike";`

M	i	k	e	\0
---	---	---	---	----

 name

(2) `return s[index];`

공간에 대한
참조, 즉 이름
의 이름 리턴

M	i	k	e	\0
---	---	---	---	----

s[index]

(3) `find(name, 0) = 'S';`

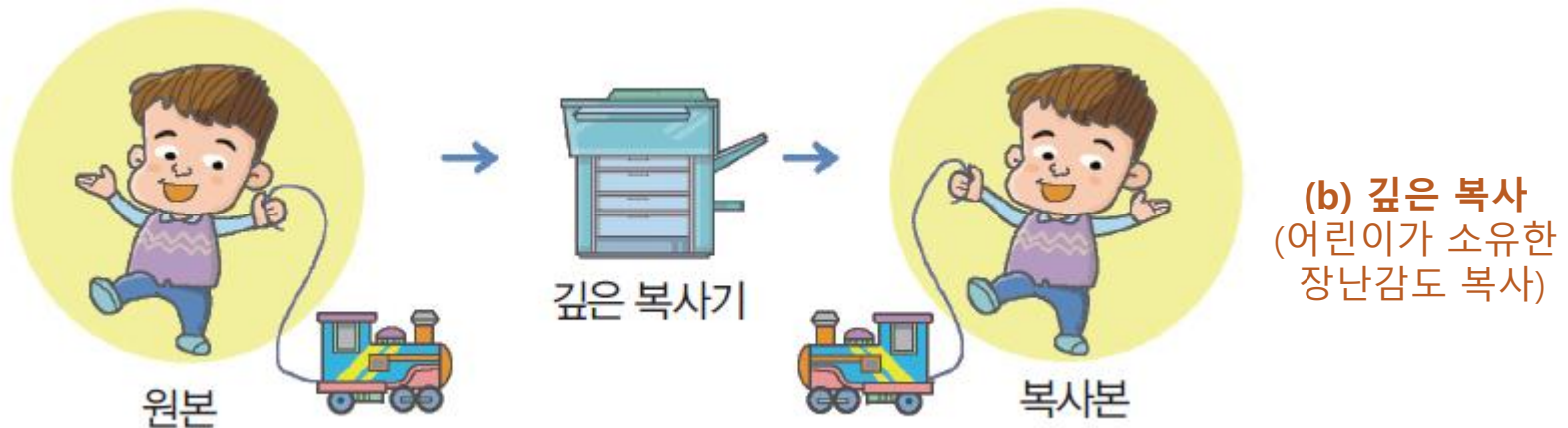
S	i	k	e	\0
---	---	---	---	----

(4) `ref = 't';`

S	i	t	e	\0
---	---	---	---	----

Mike
Sike
Site

얇은 복사와 깊은 복사



C++에서 얇은 복사와 깊은 복사

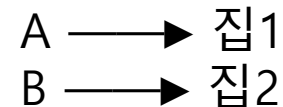
얇은 복사(Shallow Copy)

- 같이 쓰기
- 복사를 했는데 하나를 같이 쓰는 상황
- 주소만 복사
- 하나 바꾸면 같이 바뀜



깊은 복사(Deep Copy)

- 따로 쓰기
- 복사를 해서 따로 쓰는 상황
- 새로 만들어서 복사
- 하나 바꾸어도 영향이 없음



C++에서 얇은 복사와 깊은 복사

얇은 복사

```
#include <iostream>

int main() {
    int* a = new int(5);
    int* b = new int(3);

    a = b;

    delete a;
    delete b;
}
```

깊은 복사

```
#include <iostream>

int main() {
    int* a = new int(5);
    int* b = new int(3);

    *a = *b;

    delete a;
    delete b;
}
```

오류발생

얕은 복사

중단점 명령 실행됨

중단점 명령(_debugbreak) 문 또는 유사한 호출
ConsoleApplication1.exe에서 실행되었습니다.

```
#include <iostream>
using namespace std;

class Test {
public:
    int* data;

    Test(int val) {
        data = new int(val);
    }
};

int main() {
    Test a(10);
    Test b = a;

    *b.data = 20;

    cout << *a.data << endl;
    cout << *b.data << endl;
}
```

 Microsoft

20
20

```
..
#include <iostream>
using namespace std;

class Test {
public:
    int* data;

    Test(int val) {
        data = new int(val);
    }
};

int main() {
    Test a(10);
    Test b = a;

    *b.data = 20;

    cout << *a.data << endl;
    cout << *b.data << endl;

    delete a.data;
    delete b.data;
}
```


깊은 복사

```
#include <iostream>
using namespace std;

class Test {
public:
    int* data;

    Test(int val) {
        data = new int(val);
    }
    Test(const Test& t) {
        data = new int(*(t.data)); //새 메모리 만들고 값복사
    }
};

int main() {
    Test a(10);
    Test b = a; // 깊은 복사

    *b.data = 20;

    cout << *a.data << endl;
    cout << *b.data << endl;
}
```



Microsoft

10

20

복사 생성자

- 복사 생성자(copy constructor)란?
 - ▣ 객체의 복사 생성시 호출되는 특별한 생성자
- 특징
 - ▣ 한 클래스에 오직 한 개만 선언 가능
 - ▣ 복사 생성자는 보통 생성자와 클래스 내에 중복 선언 가능
 - ▣ 모양
 - 클래스에 대한 참조 매개 변수를 가지는 독특한 생성자
- 복사 생성자 선언

```
class Circle {  
    .....  
    Circle(const Circle& c); // 복사 생성자 선언  
    .....  
};  
  
Circle::Circle(const Circle& c) { // 복사 생성자 구현  
    .....  
}
```

자기 클래스에 대한
참조 매개 변수

예제 5-9 Circle의 복사 생성자와 객체 복사

```
#include <iostream>
using namespace std;

class Circle {
private:
    int radius;
public:
    Circle(const Circle& c); // 복사 생성자 선언
    Circle() { radius = 1; }
    Circle(int radius) { this->radius = radius; }
    double getArea() { return 3.14*radius*radius; }
};

Circle::Circle(const Circle& c) { // 복사 생성자 구현
    this->radius = c.radius;
    cout << "복사 생성자 실행 radius = " << radius << endl;
}

int main() {
    Circle src(30); // src 객체의 보통 생성자 호출
    Circle dest(src); // dest 객체의 복사 생성자 호출

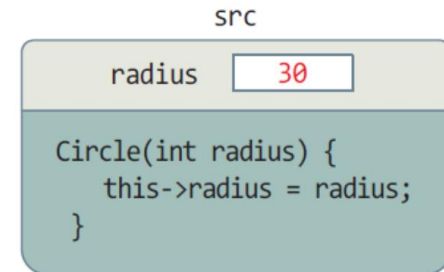
    cout << "원본의 면적 = " << src.getArea() << endl;
    cout << "사본의 면적 = " << dest.getArea() << endl;
}
```

dest 객체가 생성될 때
Circle(const Circle& c)

복사 생성자 실행 radius = 30
원본의 면적 = 2826
사본의 면적 = 2826

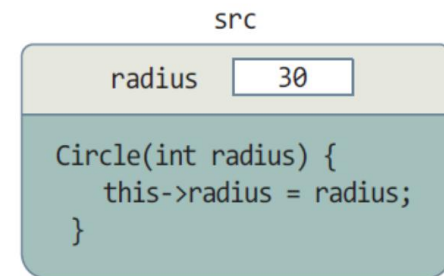
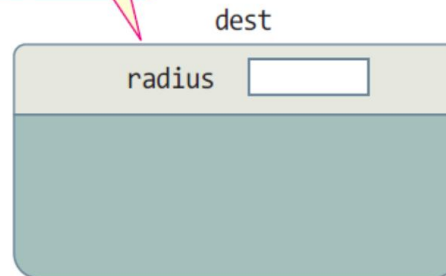
복사 생성 과정

(1) `Circle src(30);`



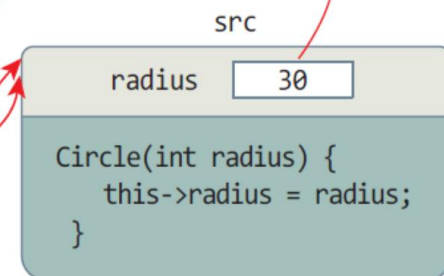
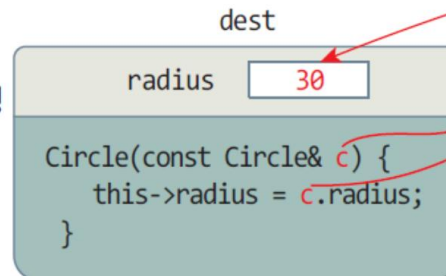
(2) `Circle dest(src);`

dest 객체
공간 할당



전달

(3) dest 객체의 복사 생성자
`Circle(const Circle& c)` 실행



복사